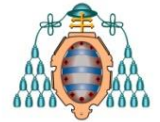




School of Computing Engineering



UNIVERSIDAD DE OVIEDO

INTRODUCTION TO ECLIPSE

Programming Methodology

This is about ...

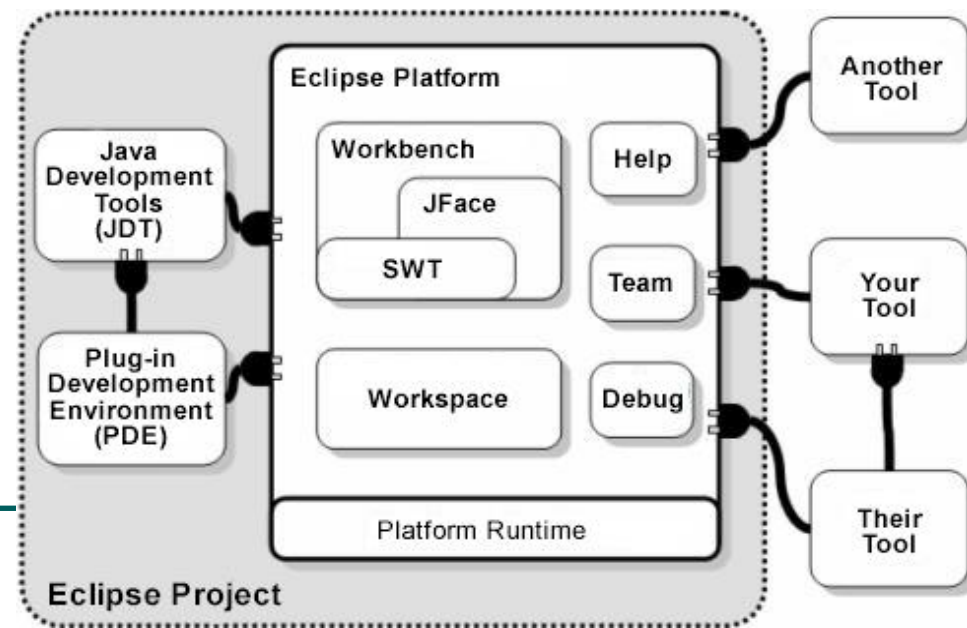
- Eclipse A new (professional) development tool to replace your old Java IDE
 - Basic concepts package, project, import/export, main
 - Tasks
 - Create a new project from scratch
 - Create project, classes, ...
 - Import an existing project
 - Export project
 - Homework
-

Architecture

- Eclipse SDK written in Java = Platform + plugins

- **Plugin** is the smallest deployable and installable software component of Eclipse.
 - Each plugin **extends** default Eclipse with some functionality.
- **PDE** (Plug-in development environment) provides a number of views and editors that make is easier to build plugins for Eclipse.

- Java IDE = Eclipse platform + Java development components (e.g. JDT)
- C++ IDE = Eclipse platform + C/C++ development components (e.g. CDT)



Download Eclipse

- Download for free [here](#)
- Available for different OS
- New releases each year

Download Links

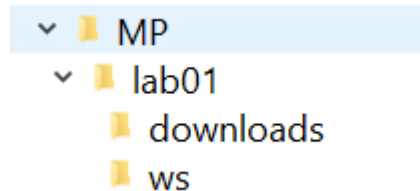
Windows x86_64
macOS x86_64 | AArch64
Linux x86_64 | AArch64

- Other builds
- Eclipse 2022-12 (4.26)
- Eclipse 2022-09 (4.25)
- Eclipse 2022-06 (4.24)
- Eclipse 2022-03 (4.23)
- Eclipse 2021-12 (4.22)
- Eclipse 2021-09 (4.21)
- Eclipse 2021-06 (4.20)
- Eclipse 2021-03 (4.19)
- Older Versions

- Download, uncompress

Projects organization and naming

- Top level folder: **MP**
- Subfolder **labID**, one for each session: **lab01**
- Subfolders
 - **downloads** save here any stuff downloaded from Campus Virtual or other source
 - **ws** workspace for the laboratory session
- **Project names** (notice small letters, no accent marks, underscore)
surname1_surname2_name_lab01_projectName
- **Package names** Starting with **uo.mp**. Then **laboratory session id**, project name, package name
uo.mp.lab01.game.model
- Last name depends on the functionality, usually names as model, service, ui, etc.
- Depending on the functionality of the code in the package



Workbench – Work environment

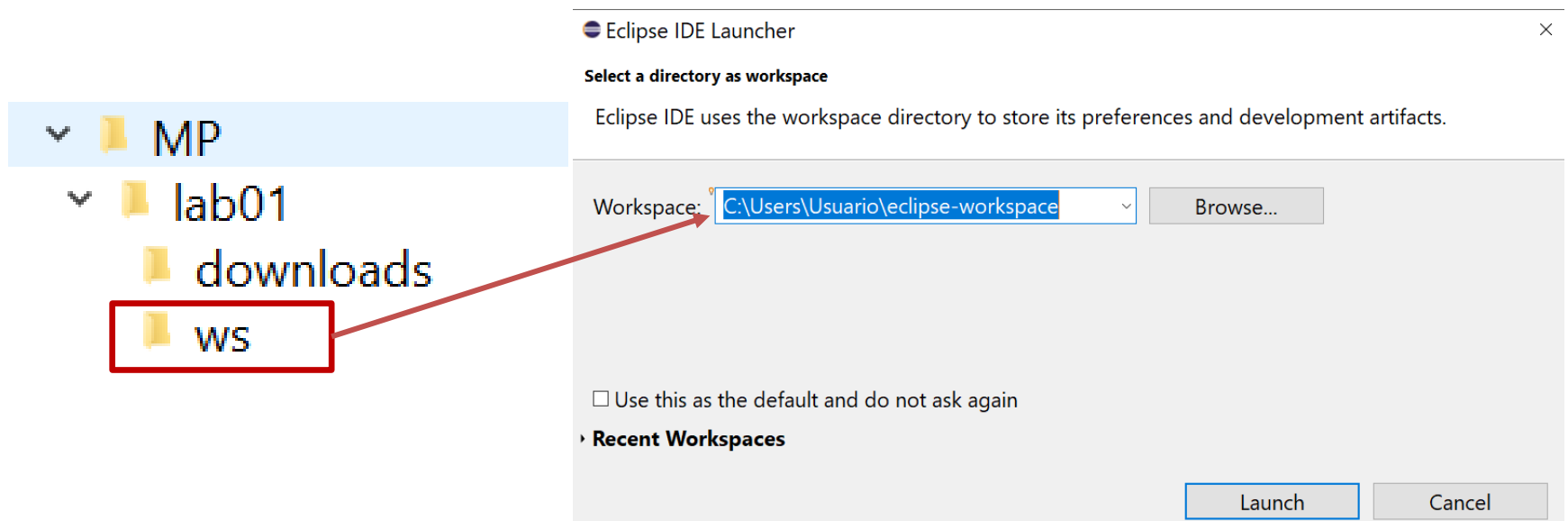
- The *workbench* is the environment in which all of your work will be done
- It provides the **user interface**
 - Menu bar and tool bars
 - Explorer,
 - editor,
 - compiler, debugger, ...
- Also includes **tools** for helping the programmer
 - Refactoring
 - Code generation
 - Testing

Workspace

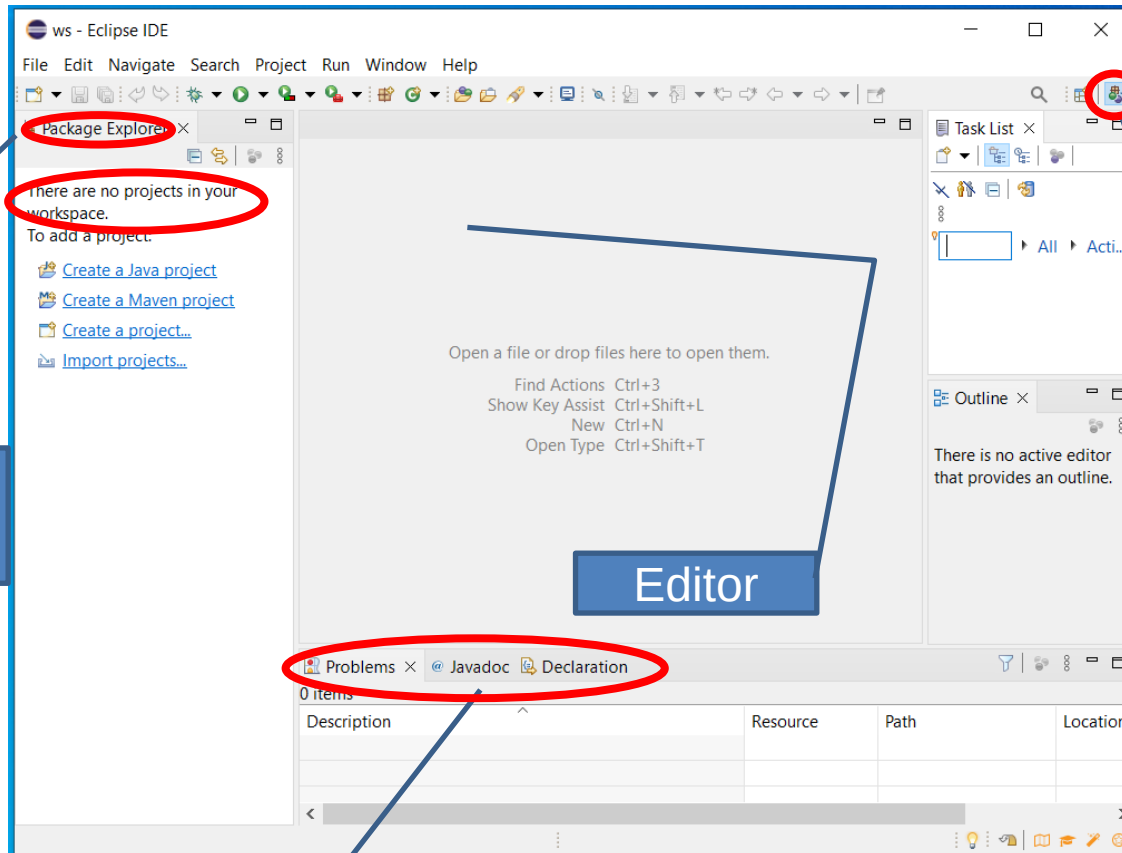
- A **workspace** is a **folder where projects are stored**
 - **Project** is a set or related resources to solve a problem (source code, settings, libraries, configuration files, ...)
 - All together save the state of the project and user settings
- You can create as many workspaces as you need
- **Use a different workspace for each laboratory class**
- **Save your projects at the end of each laboratory session**
 - Dropbox, OneDrive, ...
 - **but don't do it in whatever way**

Run Eclipse

- Run the Eclipse IDE
- Select ws as the workspace



Environment



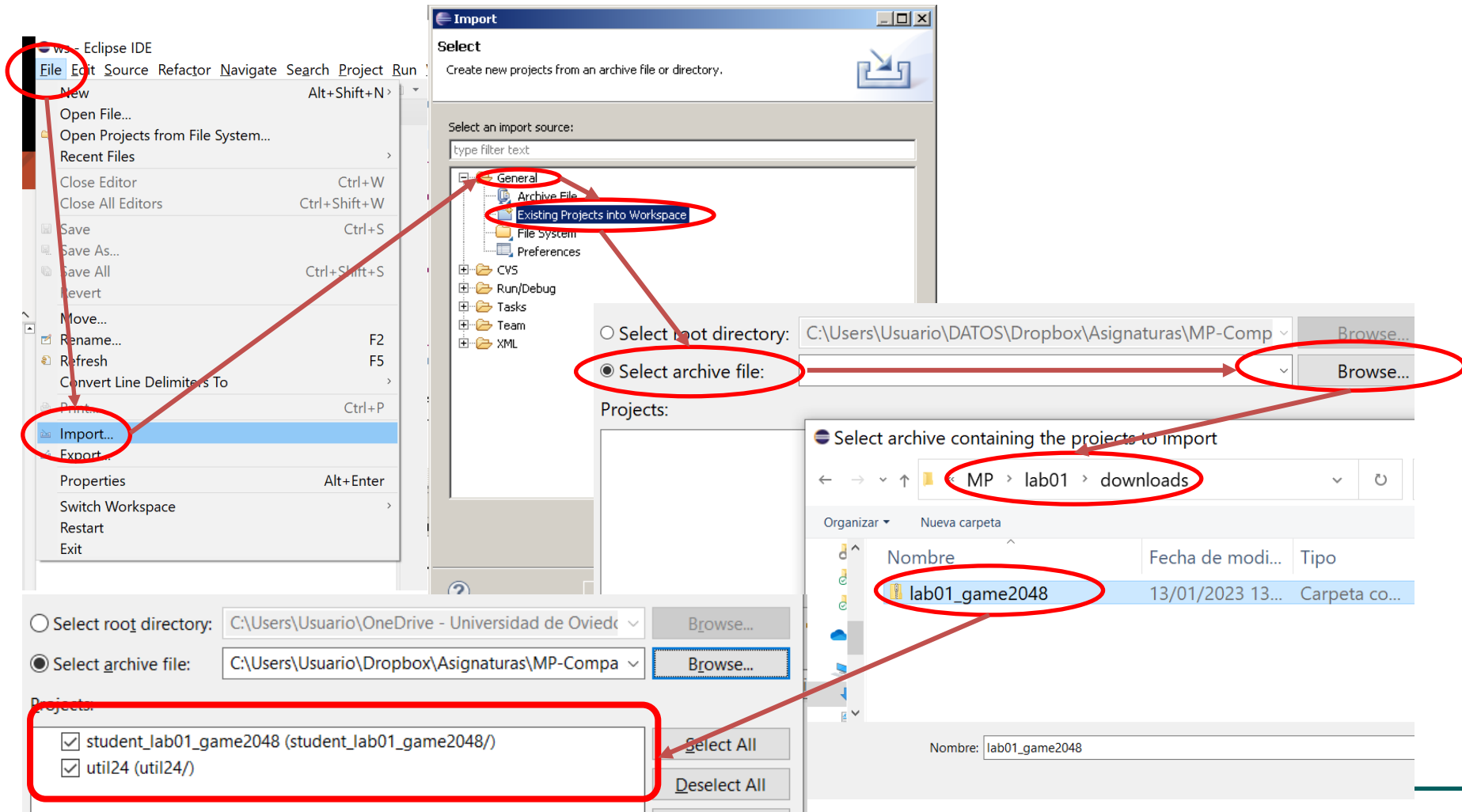
Package Explorer

Editor

Perspectives

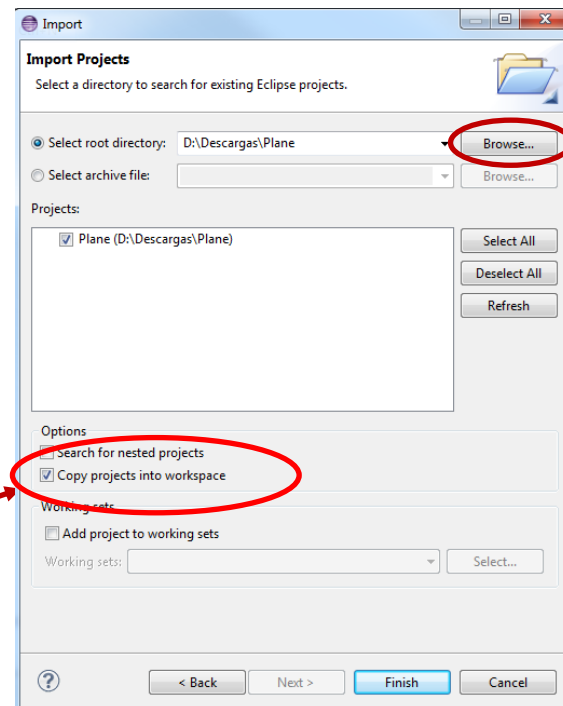
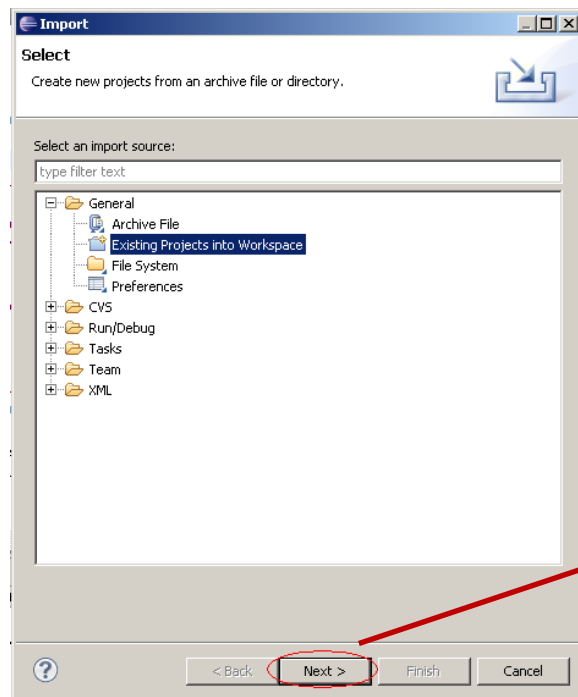
Views

Import a project



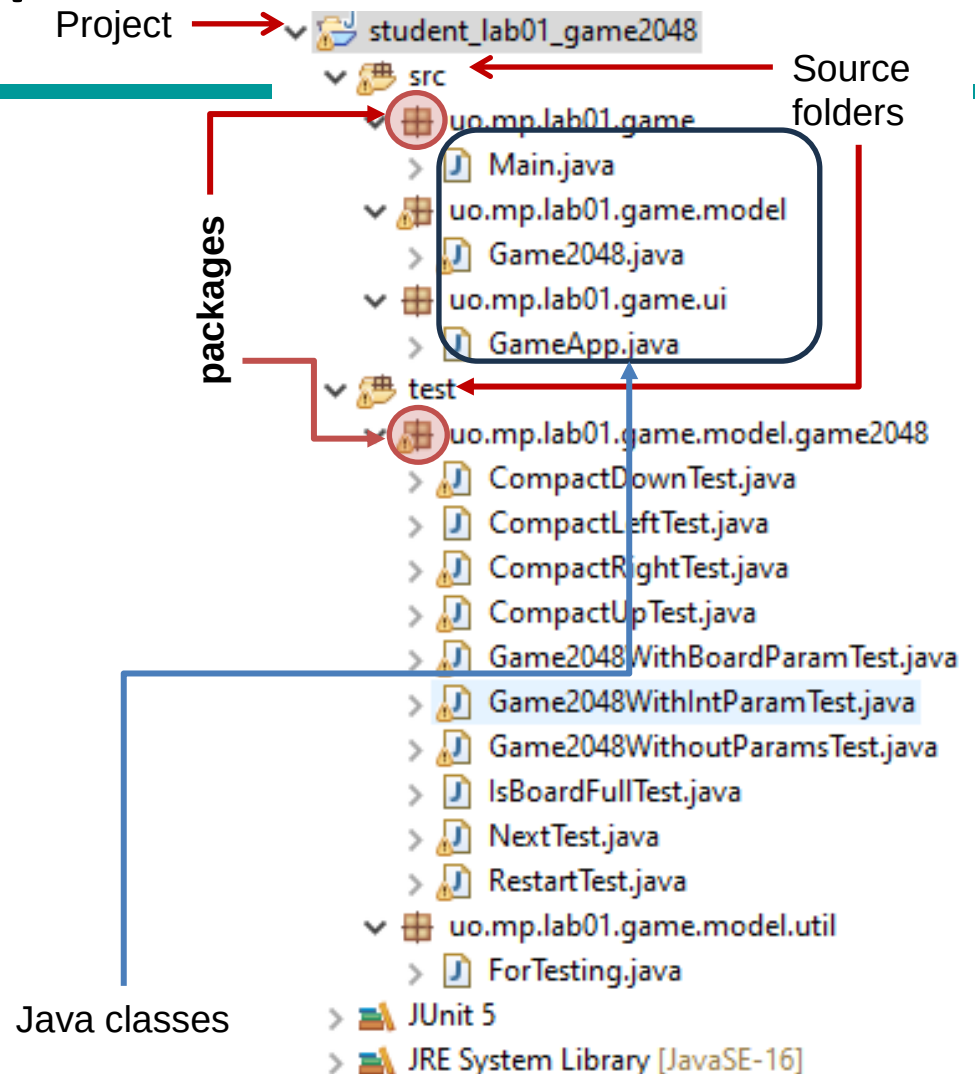
Import a project

- If the **project is not in an archive**, but in a folder, **do a copy** into current workspace.
- When importing the compressed folder, it is automatically done



Package Explorer

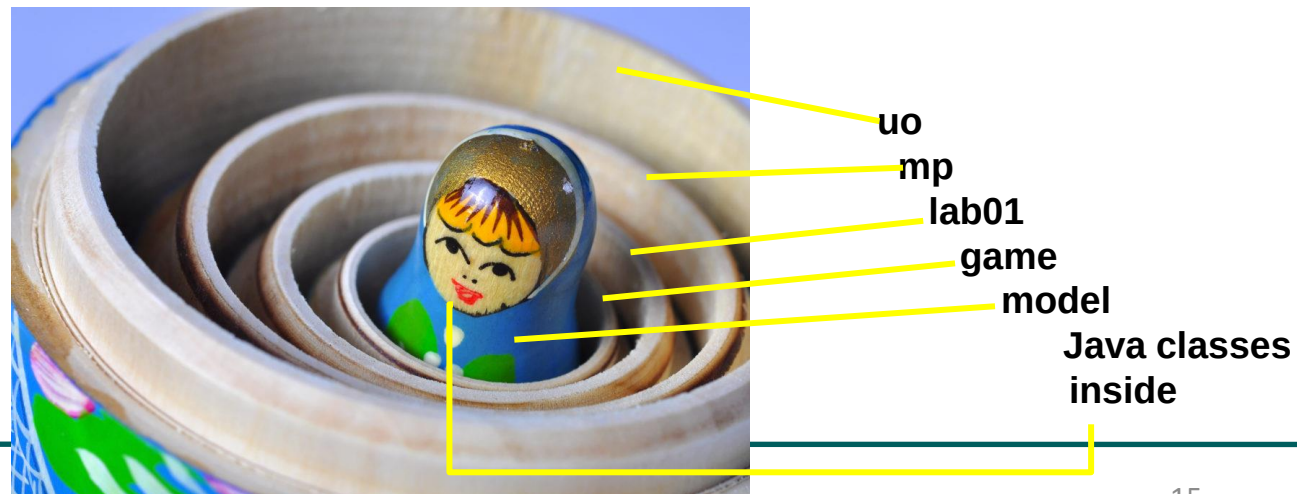
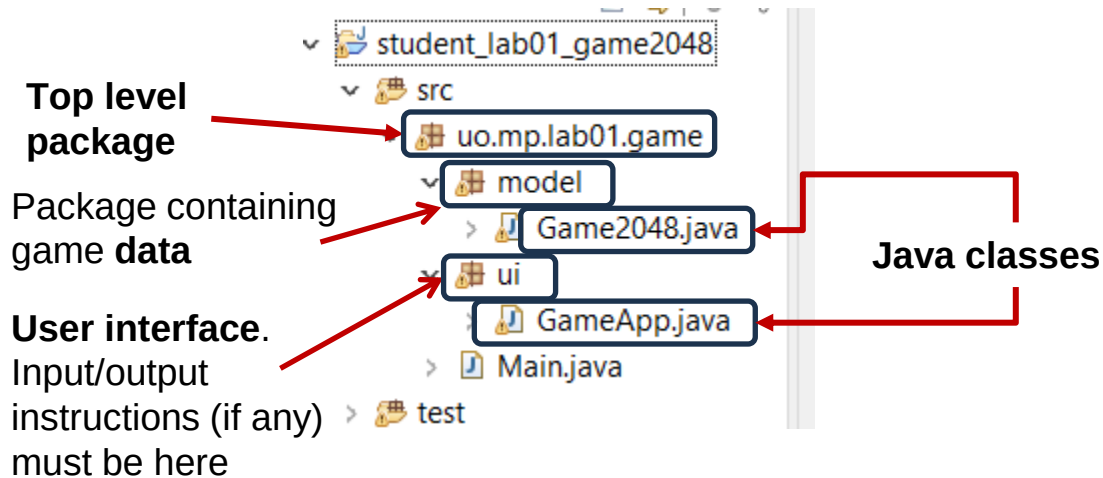
- Shows the contents of current **workspace**
 - It contains one or several **projects**
 - Each project has several **packages** and **libraries**
 - Packages contain **types**
- It is automatically synchronized with the file system
- Press F5 for resynchronization



Package

- In Java, a **package** is a **container of related types** (classes, subclasses, interfaces, enumerations and annotations) and sub-packages.
 - for example, they are focused on a common function (interface with the user, business logic, interface with databases, ...)
- Packages also provide **access protection**
 - **Access modifier** decides the accessibility of class members across the different packages (protected, default)
- and **namespace** management
 - refers to the concept of preventing naming collisions.
 - **import** statement is for namespace management.
 - The programmer can refer to a class by its name rather than by its full name including the package it is within. The compiler later substitutes the shorthand for the full class names.

Organize the code in packages



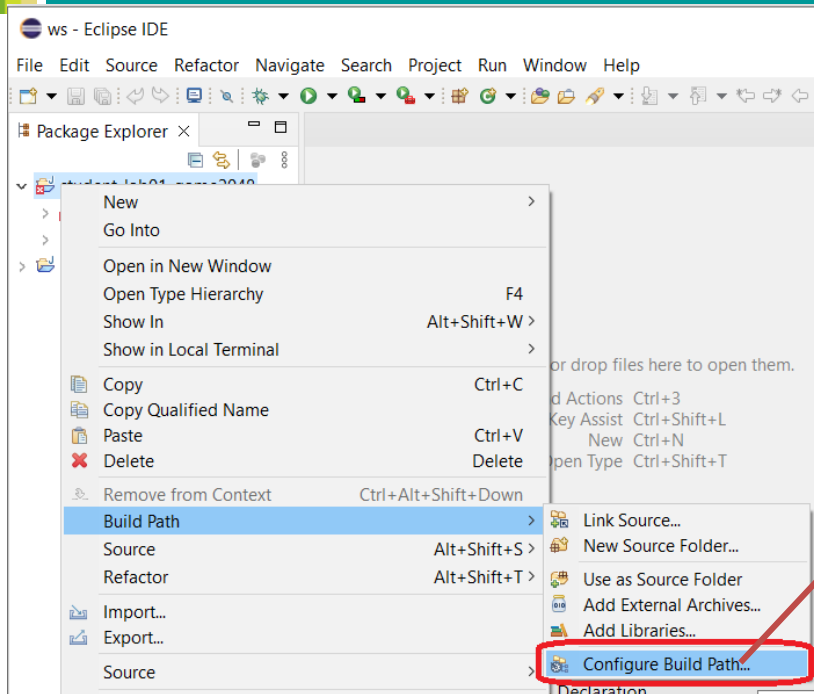
Configure the project

- Often, a project needs some piece of code that is in a library
- Be sure your Project uses **JavaSE-16**

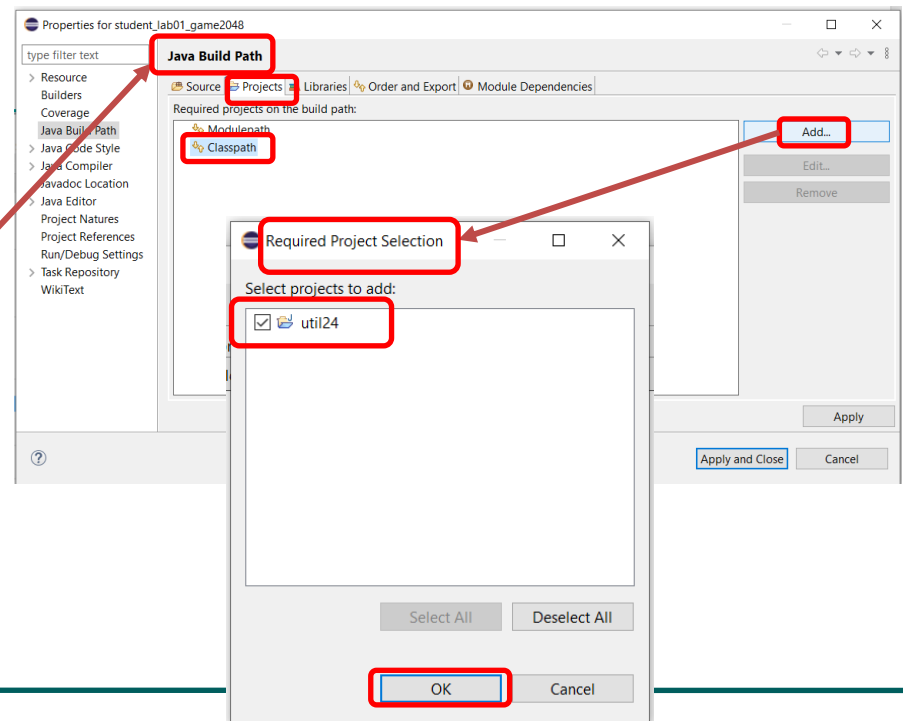
The following steps illustrate how to configure the project's build path in Eclipse IDE:

- Right-click on the project in the Package Explorer and select **Configure Build Path...**.
- In the **Properties for student_lab01_game2048** dialog, select the **Libraries** tab. The **Classpath** section is highlighted.
- Click **Add Library...** in the right-hand pane.
- In the **Add Library** dialog, select **JRE System Library**.
- In the **JRE System Library** dialog, select **Execution environment: CDC-1.0/Foundation-1.0 (jre)**. The list of JREs shows **JavaSE-16 (jre)** selected.

Configure the project



- Sometimes, a project needs some piece of code that is in **another project**
- **util**



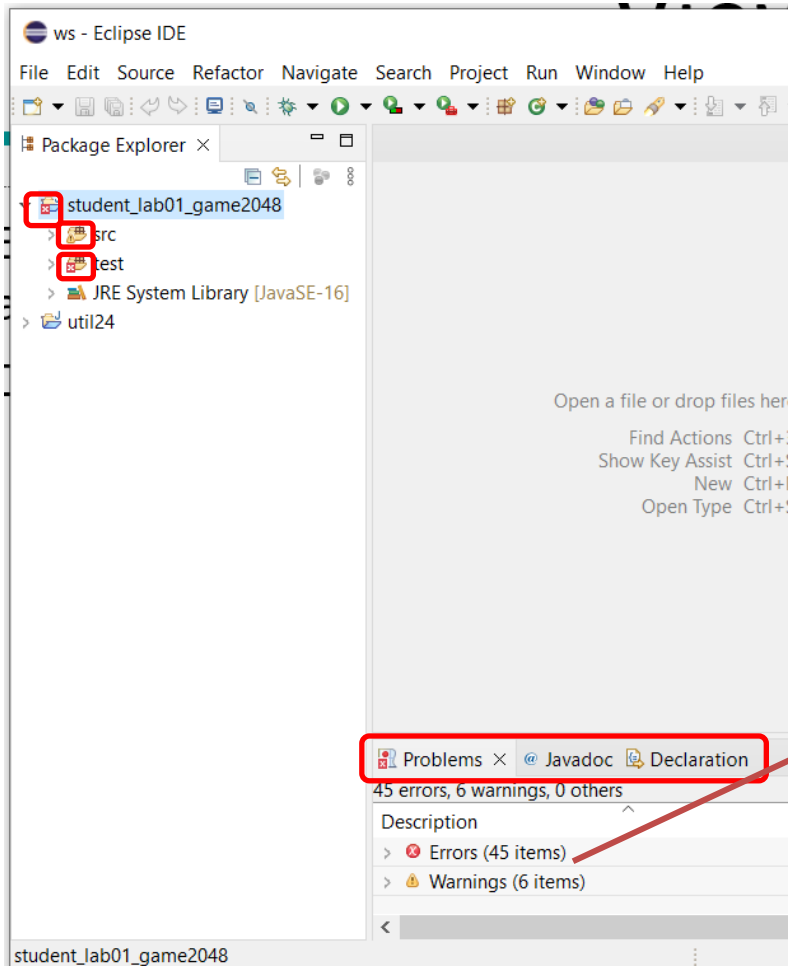
Configure the project

- Often, a project needs some piece of code that is in a library (Java, Junit)
- Or even in a different project.

The following steps illustrate how to configure a project in Eclipse IDE:

- Right-click the project in the Package Explorer and select **Build Path** > **Configure Build Path...**.
- In the **Properties for student_lab01_game2048** dialog, select the **Libraries** tab and click **Add Library...**.
- In the **Add Library** dialog, select **JRE System Library**.
- In the **JRE System Library** dialog, select **Execution environment** and choose **JavaSE-16 (jre)** from the list.

Views



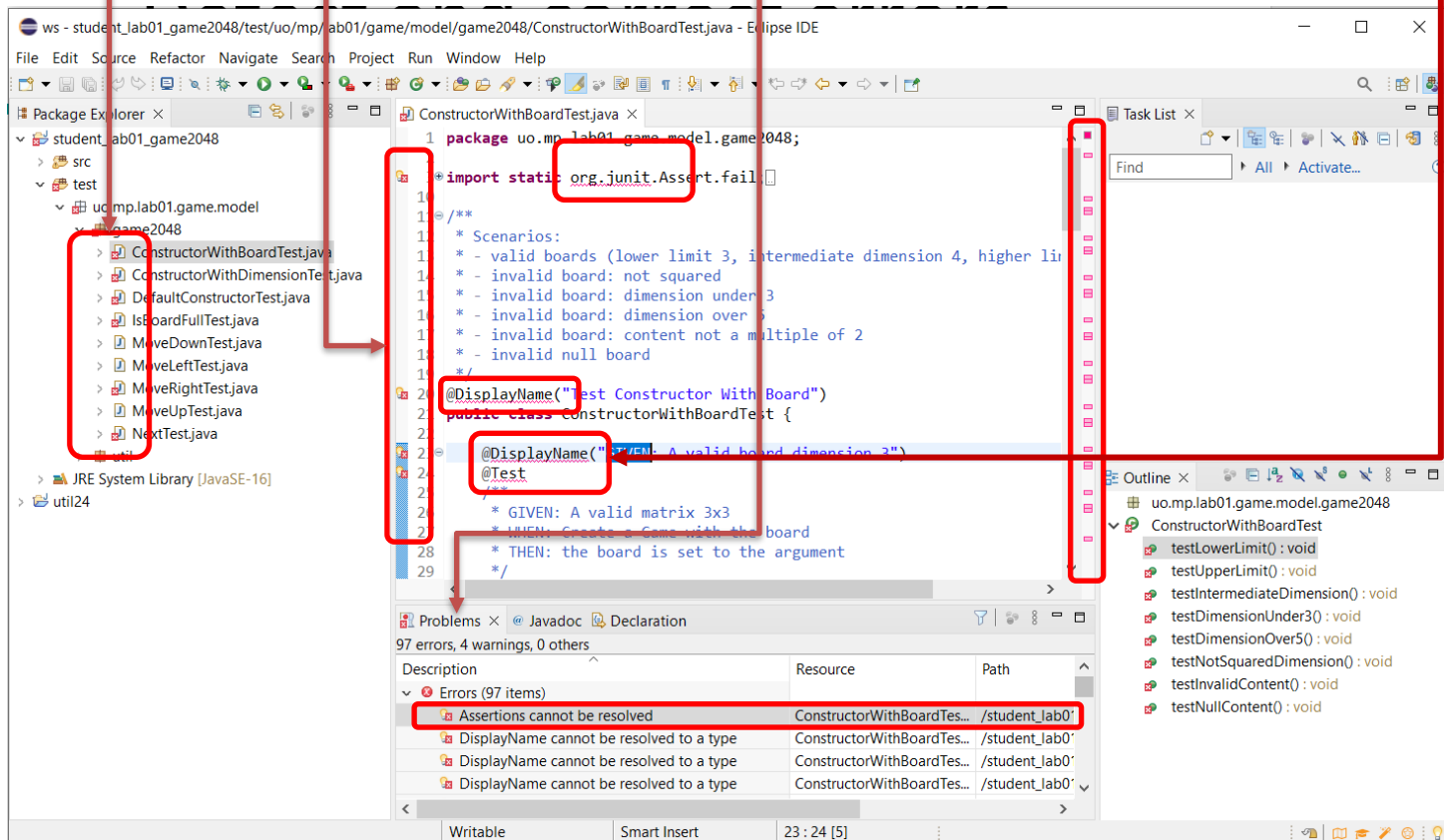
- Eclipse views provide information about the project
- Different views means different info
 - Errors / Warnings
 - The runtime console
 - Test results
 - Javadoc

The screenshot shows the Problems view with a table of errors. A red box highlights the table header. A red arrow points from the 'Problems' tab in the bottom left to the table.

Description	Resource	Path	Location	Type
97 errors, 4 warnings, 0 others				
Errors (97 items)				
Assertions cannot be resolved	ConstructorWithBoardTes...	/student_lab01_game...	line 115	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 20	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 23	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 34	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 46	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 58	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 70	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 82	Java Problem
DisplayName cannot be resolved to a type	ConstructorWithBoardTes...	/student_lab01_game...	line 84	Java Problem

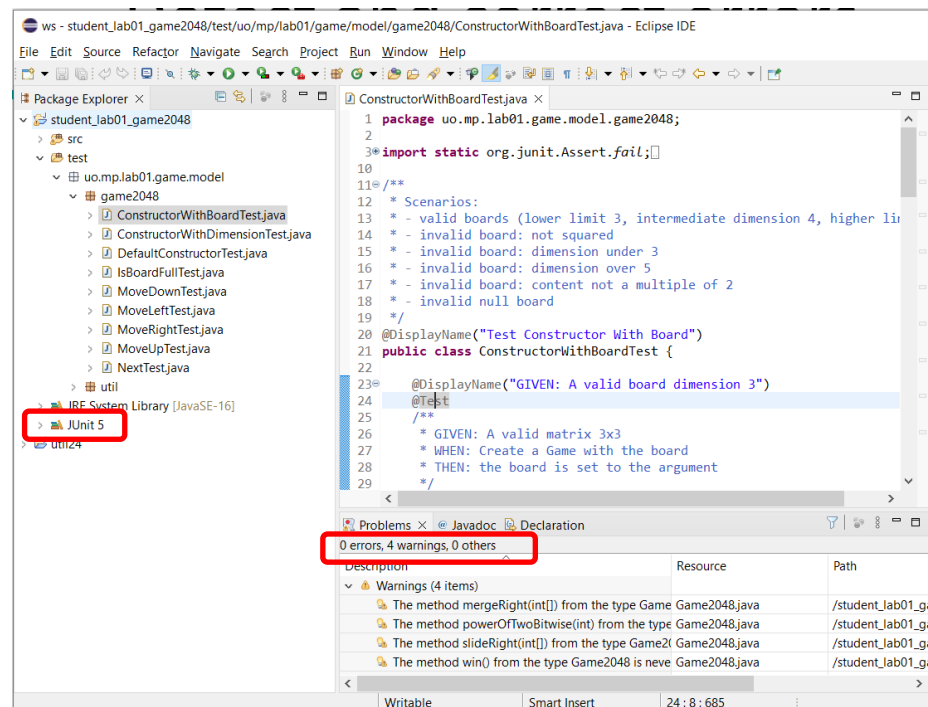
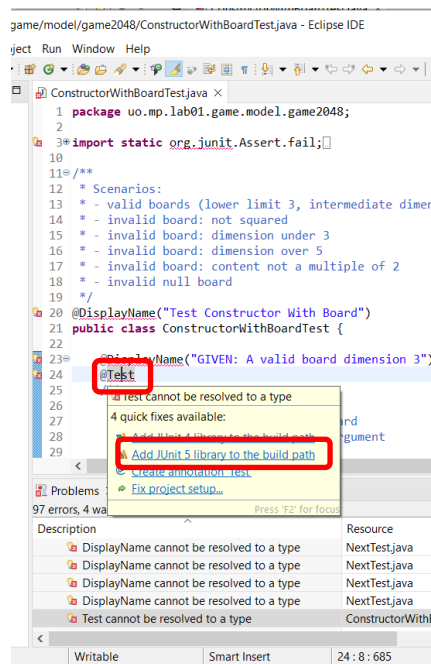
Detect and correct errors

- Errors are marked in the **Problems view**, in the **resource** itself by a curly red underline (yellow for warnings) and Eclipse also signals the problem using a **crossed bulb**.



Detect and correct errors

- By placing the cursor over the underlying line, Eclipse suggests some possibilities to correct the error
 - If one is selected, project changes trying to repair the error

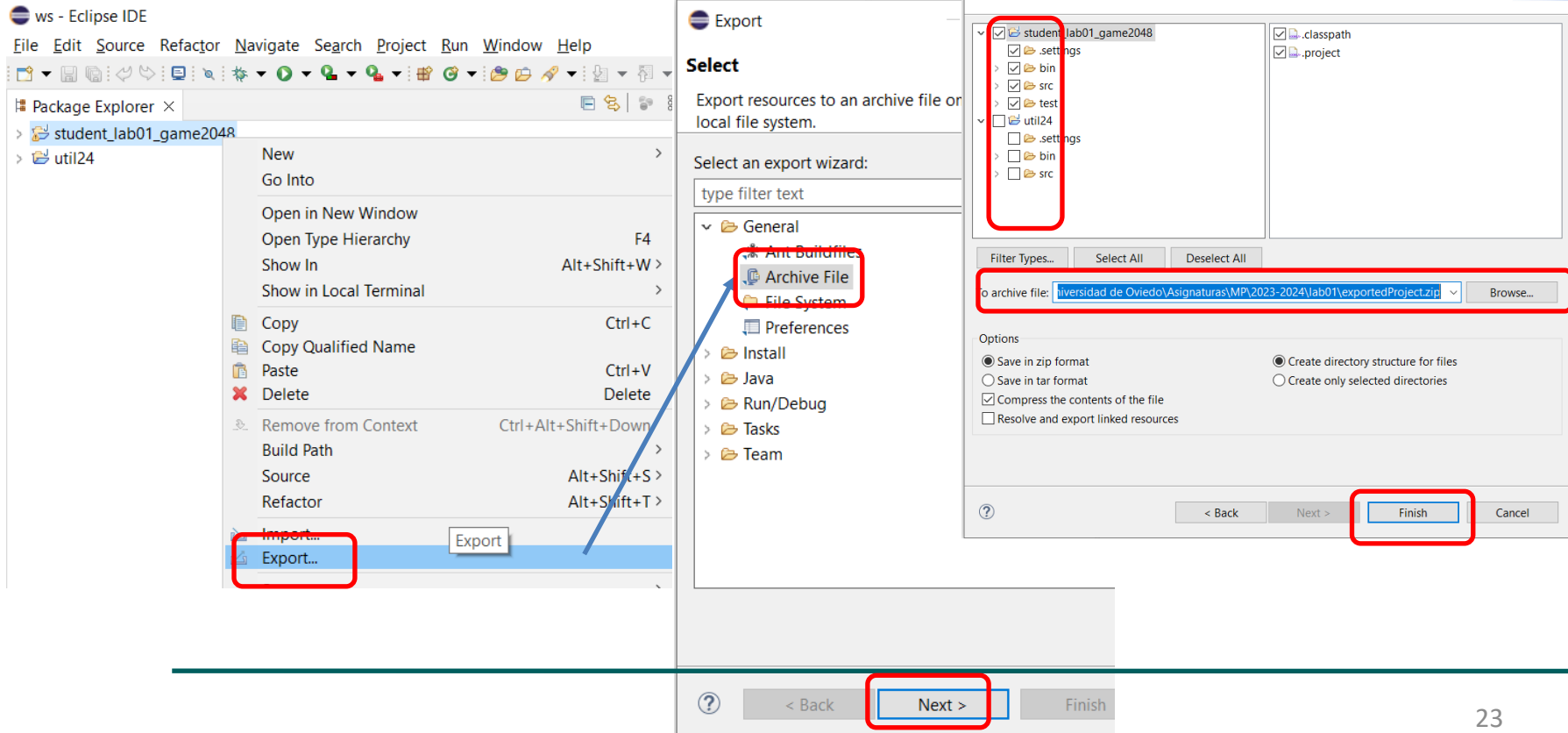


- Check Build Path

EXPORT A PROJECT

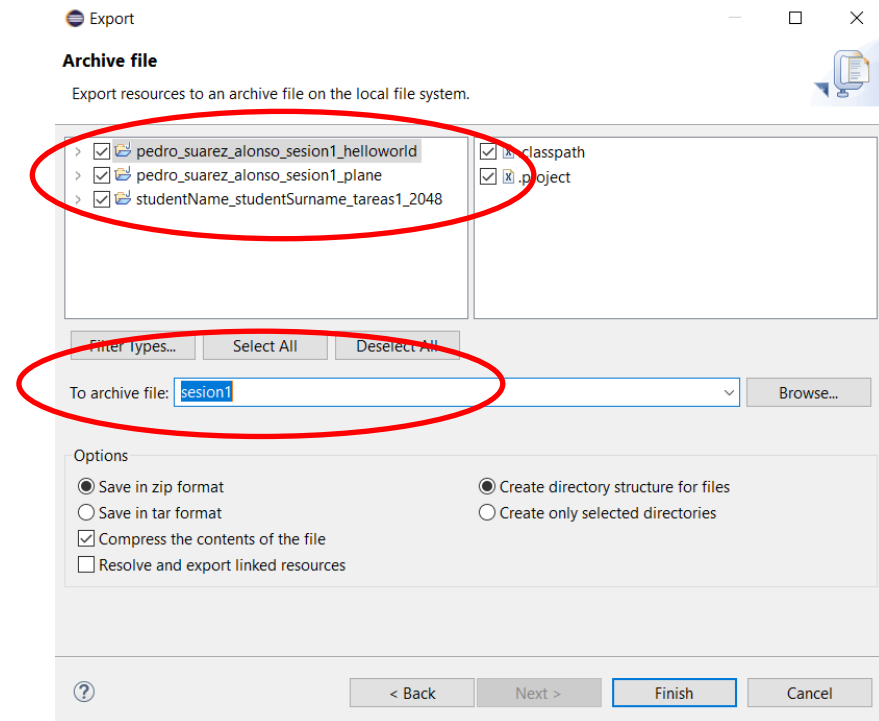
Export a project or projects

- It will create a compressed file containing the project
- File → Export or click with the right button



Export all

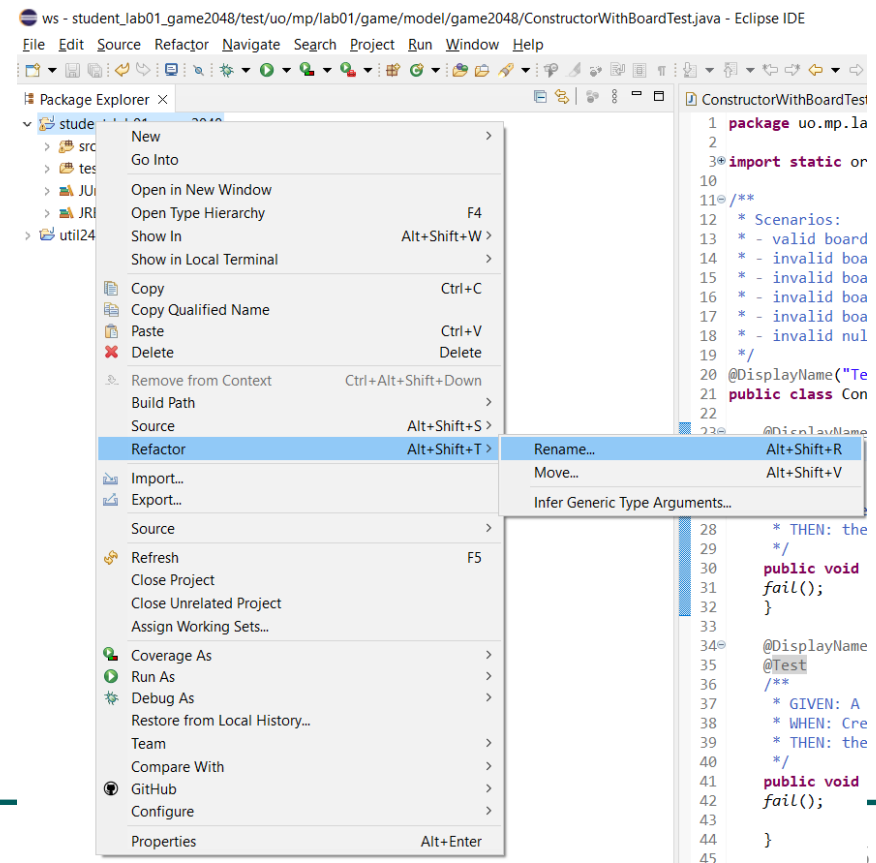
- When session is over
 - Export **ALL** projects together in a single compressed file **lab01**
 - Save this file somewhere
- At home
 - Create folder MP\lab01\ws
- Run eclipse, use ws as workspace
- Import exported project lab01.zip



Rename a project (or package, ...)

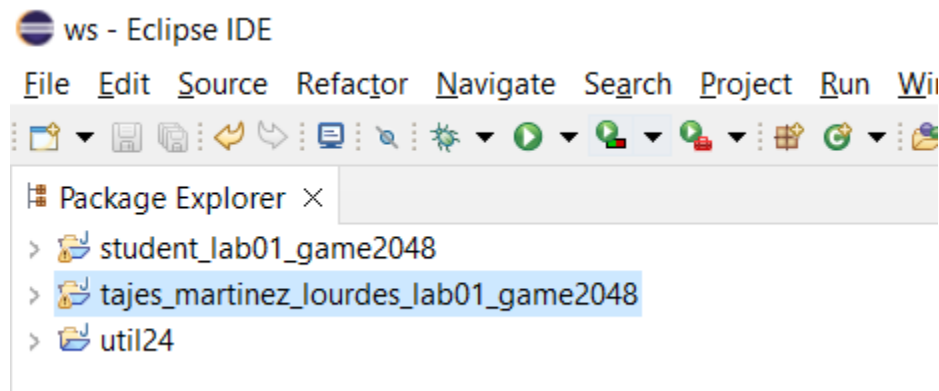
surname1_surname2_name_lab01_game2048

- Click on the project with the right button and select **Refactor/rename**
- Or **Alt+Shift+R**
- Or **F2**



TO DO

IMPORT THE EXPORTED PROJECT



Run the project

The screenshot shows the Eclipse IDE interface with the following elements highlighted:

- Run Button:** A red circle highlights the green play button in the top toolbar.
- Project:** A red box highlights the project name `tajes_martinez_lourdes_lab01_game2048` in the Package Explorer.
- Class:** A red box highlights the `Main` class in the Package Explorer.
- Method:** A red box highlights the `main(String[] args) : void` method in the Package Explorer.
- Source Editor:** A red box highlights the `public static void main(String[] args) {` line in the `Main.java` file.
- Console:** A red box highlights the `Console` window at the bottom, which shows the output: `<terminated> Main [Java Application] C:\Users\Usuario\p2\pool\plugins\`.

Red arrows point from the labels to the corresponding elements in the IDE:

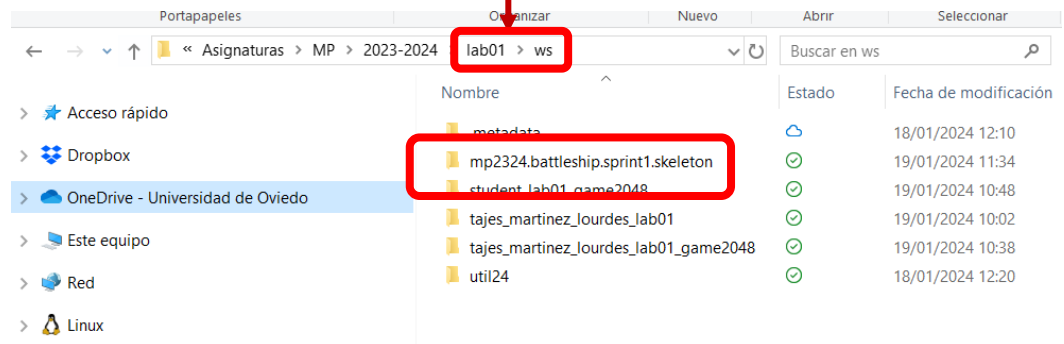
- Method:** Points to the `main` method in the source code.
- Class:** Points to the `Main` class in the Package Explorer.
- File:** Points to the `Main.java` file in the Package Explorer.
- Project:** Points to the `tajes_martinez_lourdes_lab01_game2048` project in the Package Explorer.

Run the class with main method
No main() means no run
Try to run other class

Output, if any, will be here

TO DO

1. IMPORT THE BATTLESHIP PROJECT
2. TAKE A LOOK TO MP->lab01->ws
3. FIX THE ERRORS
4. RUN



Run the project

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure, with `mp2324.battleship.sprint1.skeleton` and `Main.java` highlighted. The main editor shows the `Main.java` file with the following code:

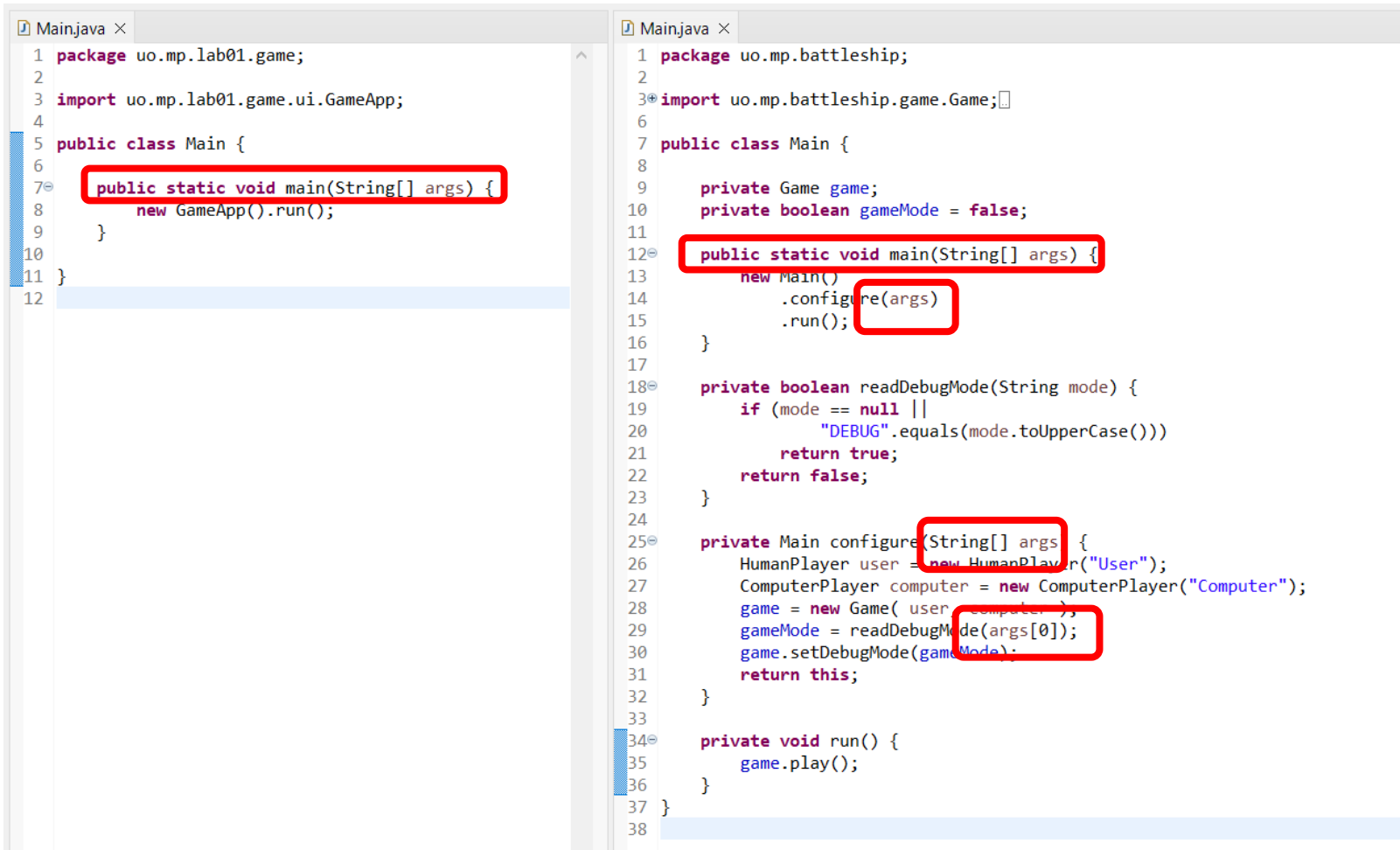
```
1 package uo.mp.battleship;
2
3 import uo.mp.battleship.game.Game;
4
5
6
7 public class Main {
8
9     private Game game;
10    private boolean gameMode = false;
11
12    public static void main(String[] args) {
13        new Main()
14            .configure(args)
15            .run();
16    }
17
18    private boolean readDebugMode(String mode) {
19        if (mode == null ||
20            "DEBUG".equals(mode.toUpperCase()))
```

The `main` method is highlighted, and the `args` parameter is also highlighted. The Console window at the bottom shows the following error message:

```
<terminated> Main (1) [Java Application] C:\Users\Usuario\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full...
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 0 out of
at uo.mp.battleship.Main.configure(Main.java:29)
at uo.mp.battleship.Main.main(Main.java:14)
```

The error message is highlighted, indicating a runtime exception.

Main::main(String[] args)



```
1 package uo.mp.lab01.game;
2
3 import uo.mp.lab01.game.ui.GameApp;
4
5 public class Main {
6
7     public static void main(String[] args) {
8         new GameApp().run();
9     }
10
11 }
12
```

```
1 package uo.mp.battleship;
2
3 import uo.mp.battleship.game.Game;
4
5 public class Main {
6
7     private Game game;
8     private boolean gameMode = false;
9
10     public static void main(String[] args) {
11         new Main()
12             .configure(args)
13             .run();
14     }
15
16     private boolean readDebugMode(String mode) {
17         if (mode == null ||
18             "DEBUG".equals(mode.toUpperCase()))
19             return true;
20         return false;
21     }
22
23     private Main configure(String[] args) {
24         HumanPlayer user = new HumanPlayer("User");
25         ComputerPlayer computer = new ComputerPlayer("Computer");
26         game = new Game(user, computer);
27         gameMode = readDebugMode(args[0]);
28         game.setDebugMode(gameMode);
29         return this;
30     }
31
32     private void run() {
33         game.play();
34     }
35
36 }
37
38
```

Run the project with parameters

The screenshot illustrates the steps to run a Java application with parameters in Eclipse IDE:

- Package Explorer:** The project structure is shown on the left. The file `Main.java` is highlighted.
- Context Menu:** A right-click context menu is open over `Main.java`. The `Run As` option is selected.
- Run Configurations Dialog:** The `Run Configurations` dialog is open, showing the configuration for `Main (1)`. The `Program arguments` field contains the text `debug`.
- Run Button:** The `Run` button is highlighted.

The `Run Configurations` dialog shows the following details:

- Name:** Main (1)
- Program arguments:** debug
- VM arguments:**
- ☒ Use the `-XX:ShowCodeDetailsInExceptionMessages` argument when launching
- ☐ Use `@argfile` when launching
- Buttons:** Show Command Line, Revert, Apply, Run, Close

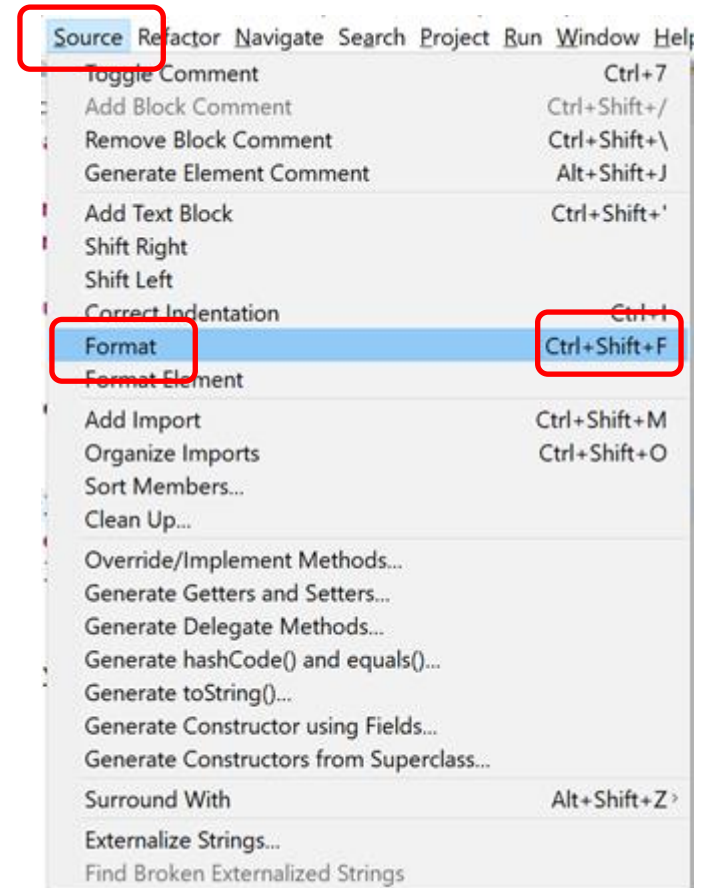
SOURCE CODE MANAGEMENT

Format source code

- Edit class GameApp in Project game2048

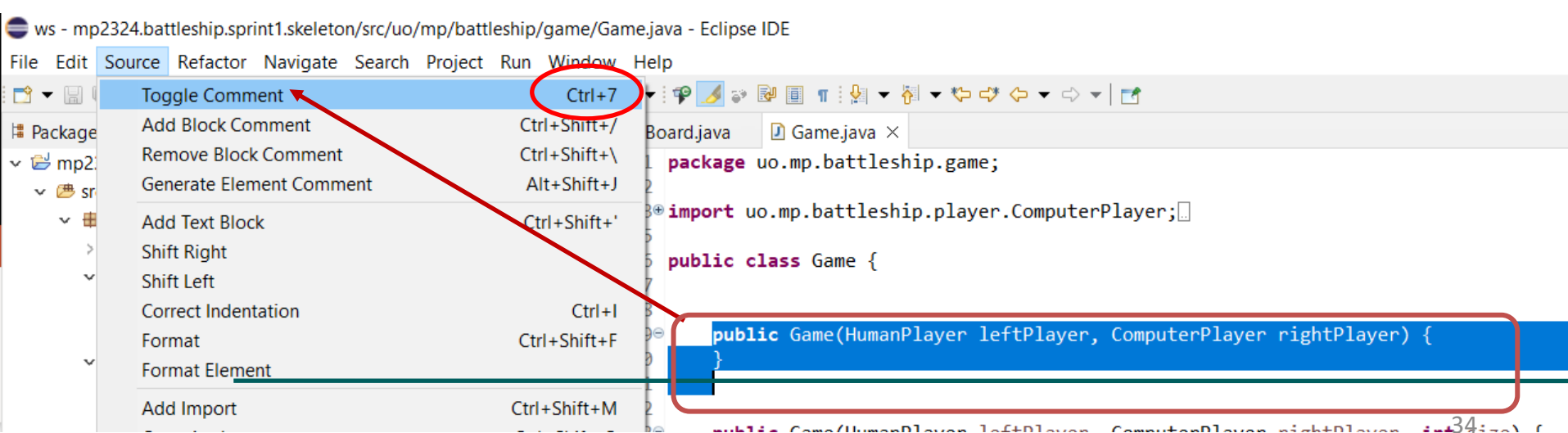


```
GameApp.java ×
1 package uo.mp.lab01.game.ui;
2
3 import uo.mp.lab01.game.model.Game2048;
4
5 /* Main class of the 2048 game */
22 public class GameApp {    private Game2048 game = new Game2048();
23
24     public void run() {
25         do {            play(); } while (userWantsAnotherPlay());
26     }
27
28     private boolean userWantsAnotherPlay() {
29         Output.showNewGamePrompt();    return Input.readCharacter() == 's';
30     }
31
32     private void play() {
33         Output.printTitle();
34
35         do {            game.next();
36             System.out.println(game.toString());    doMovement(); } while (!game.notFinished());
37         Output.showGameOver();
38     }
39
40     private void doMovement() { Output.showMovements();
41         switch (Input.readCharacter()) {
42             case 'r':    case 'R':            game.moveRight();            break;    case 'l':    case 'L':
43                 game.moveLeft();
44                 break;
45             case 'u':
46             case 'U':            game.moveUp();            break;    case 'd':    case 'D':            game.moveDown();
47                 break;
48         }
49     }
50
51 }
52
```



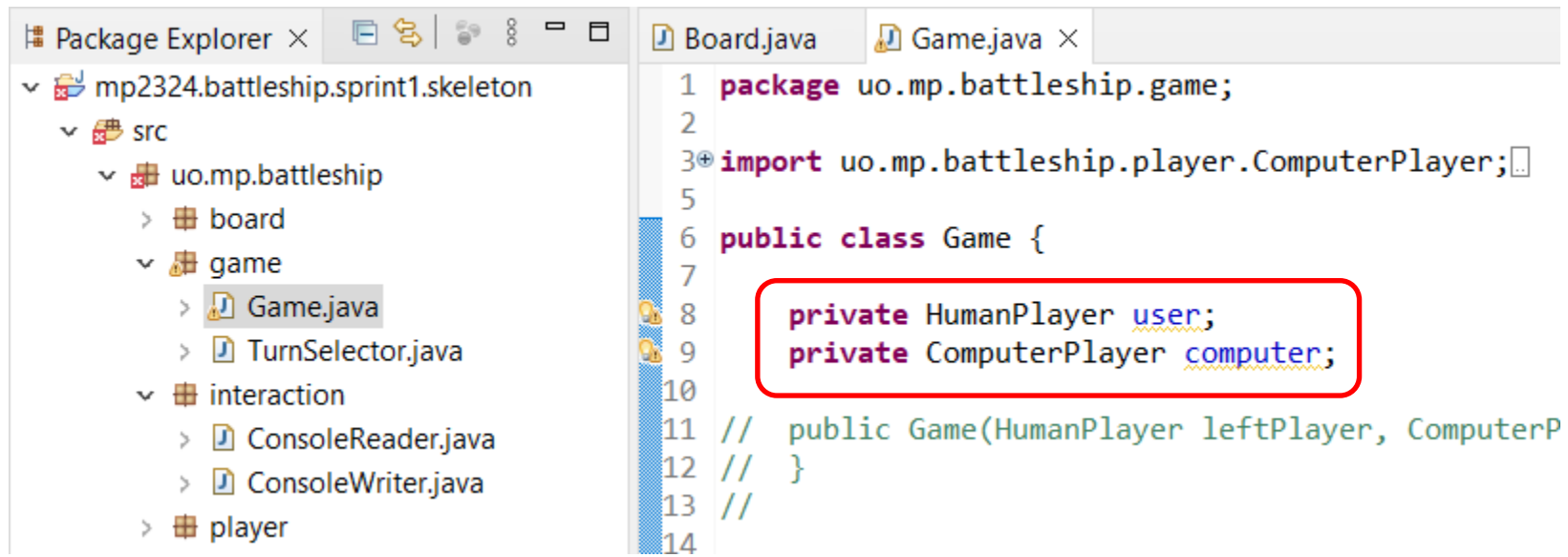
Comments Source code

- Comment or uncomment code by selecting a line or a block of code and tagging it as a comment
- Source → Toggle Comment : //
- Shortcut: **Control + 7**
- Source → Add Block Comment: /* ... */



Source code: Code generation

- Eclipse uses templates for helping you write repetitive code. See [Source](#)
- For generating getters and setters, constructors,
- Declare two private attributes (a HumanPlayer and a ComputerPlayer) in class Game



The screenshot shows the Eclipse IDE interface. On the left is the Package Explorer, and on the right is the editor window showing the source code of Game.java.

Package Explorer:

- mp2324.battleship.sprint1.skeleton
 - src
 - uo.mp.battleship
 - board
 - game
 - Game.java** (selected)
 - TurnSelector.java
 - interaction
 - ConsoleReader.java
 - ConsoleWriter.java
 - player

Game.java Source Code:

```
1 package uo.mp.battleship.game;
2
3 import uo.mp.battleship.player.ComputerPlayer;
4
5
6 public class Game {
7
8     private HumanPlayer user;
9     private ComputerPlayer computer;
10
11 // public Game(HumanPlayer leftPlayer, ComputerP
12 // }
13 //
14
```

The two private attributes, `private HumanPlayer user;` and `private ComputerPlayer computer;`, are highlighted with a red rectangle.

ws - mp2324.battleship.sprint1.skeleton/src/uo/

File Edit **Source** Refactor Navigate Search

Package Explorer

mp2324.battleship.sprint1

src

uo.mp.battleship

board

game

Game.java

TurnSelector

interaction

ConsoleReader

ConsoleWriter

player

Main.java

JRE System Library [JRE]

student_lab01_game204

tajes_martinez_lourdes

src

uo.mp.lab01.game

model

ui

Toggle Comment

Add Block Comment

Remove Block Comment

Generate Element Comment

Add Text Block

Shift Right

Shift Left

Correct Indentation

Format

Format Element

Add Import

Organize Imports

Sort Members...

Clean Up...

Override/Implement Methods...

Generate Getters and Setters...

Generate Delegate Methods...

Generate hashCode() and equals()...

Generate toString()...

Generate Constructor using Fields...

Generate Constructors from Superclass...

Surround With

Externalize Strings...

Find Broken Externalized Strings

Generate Constructor using Fields

Select super constructor to invoke:

Object()

Select fields to initialize:

☒ user

☒ computer

Select All

Deselect All

Up

Down

Insertion point:

First member

Access modifier

☒ public ☐ protected ☐ package ☐ private

☐ Generate constructor comments

☐ Omit call to default constructor super()

The format of the constructors may be configured on the [Code Templates](#) preference page.

2 of 2 selected.

?

Generate

Cancel

Alt+Shift+Z >

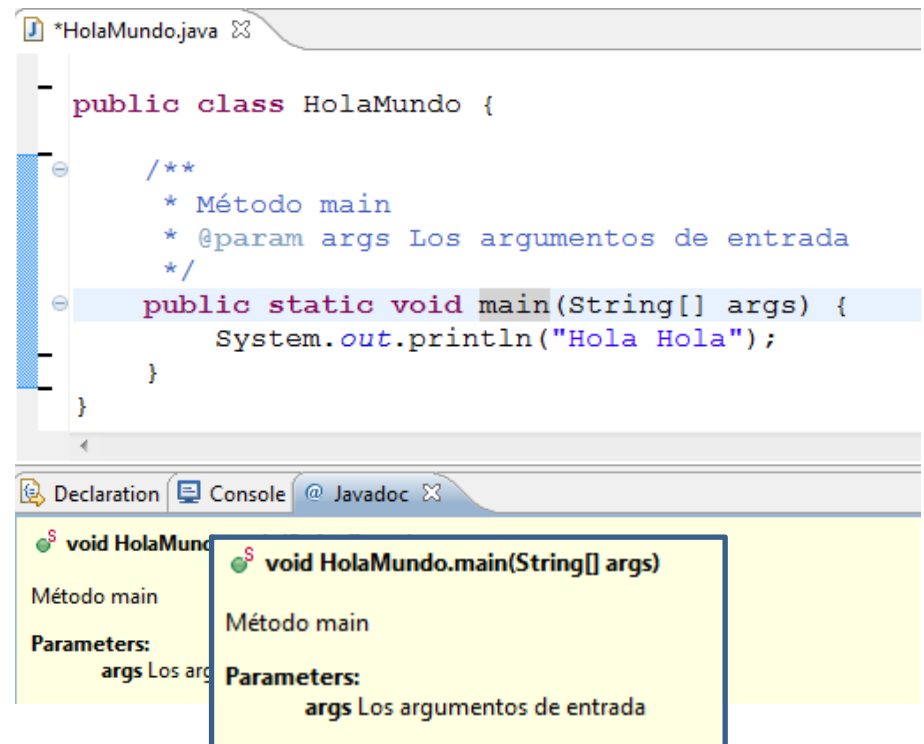
```
10 private HumanPlayer user;  
11 private ComputerPlayer computer;  
12  
13 // public Game(HumanPlayer leftPlayer, ComputerPlayer rightPlayer) {  
14 // }  
15 //  
16  
17 public Game(HumanPlayer user, ComputerPlayer computer) {  
18     super();  
19     this.user = user;  
20     this.computer = computer;  
21 }  
22
```

GENERATING JAVADOC PAGES

Javadoc comments in source code

Javadoc comments in the source code are useful

- In View Javadoc when selecting a word.
- As a hint when the cursor is over a word.
- To generate HTML pages to document the project



ws - student_lab01_game2048/src/uo/mp/lab01/game/model/Game2048.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help



Package Explorer x Main.java GameApp.java Game2048.java x

mp2324.battleship.sprint1.skeleton
student_lab01_game2048
src
uo.mp.lab01.game
model
Game2048.java
ui
GameApp.java
GameApp
game
doMovement(): void
play(): void
run(): void
userWantsAnotherPlay()
Input.java
Output.java
Main.java
test
JUnit 5
JRE System Library [JavaSE-16]
tajes_martinez_lourdes_lab01_game2048
util24

```
7  * @author MP teachers
8  * @version 1.0
9  */
10 public class Game2048 {
11
12     /**
13      * Creates a 3x3 board (default board) All cells set to 0
14      */
15     public Game2048() {
16         // TODO
17     }
18
19     /**
20      * Creates a squared board with the specified size, zeroes all its positions
21      *
22      * @param size the desired size for the board
23      */
24     public Game2048(int size) {
25         // TODO
26     }
27
28     /**
29      * Creates a squared board with the specified size, zeroes all its positions
30      *
31      * @param size the desired size for the board
32      */
33
34     public Game2048(int[][] board) {
```

Problems Javadoc x Declaration Console

uo.mp.lab01.game.model.Game2048.Game2048(int size)

Creates a squared board with the specified size, zeroes all its positions

Parameters:

size the desired size for the board

Generate Javadoc comments

It is highly recommended to write Javadoc comments in the source code

The screenshot shows an IDE window with a Java file named `Game.java`. The code includes package declarations, imports, and a `Game` class with private fields `user` and `computer`. A method `Game(HumanPlayer user, ComputerPlayer computer)` is highlighted. A context menu is open over the method signature, with the `Source` menu item selected. The `Generate Element Comment` option is highlighted in the menu. A red box highlights the method signature in the code, and another red box highlights the generated Javadoc comment and the method body. The generated Javadoc comment is as follows:

```
/**
 * @param user
 * @param computer
 */
public Game(HumanPlayer user, ComputerPlayer computer) {
    super();
    this.user = user;
    this.computer = computer;
}
```

Generate Javadoc templates

It is highly recommended to write Javadoc comments in the source code

The screenshot shows an IDE window with a Java file named `Game.java`. The code includes package declarations, imports, and a `Game` class with private fields `user` and `computer`. A red box highlights the `Game` class name. A context menu is open, with the `Source` menu item selected. The `Generate Element Comment` option is highlighted in blue. A red arrow points from this menu item to a red box containing the generated Javadoc template for the `Game` constructor. The template includes a multi-line comment with `@param` tags for `user` and `computer`, followed by the method signature and body. Another red arrow points from the `Game` class name to the `Game` constructor in the code. To the right of the template, the text "Or write /** and hit enter" is displayed.

```
package uo.mp.battleship.gam
import uo.mp.battleship.play
public class Game {
    private HumanPlayer user;
    private ComputerPlayer c
    // public Game(HumanPlayer
    // }
    //
    public Game(HumanPlayer u
        super();
        this.user = user;
        this.computer = compu
    }
}
```

Source Refactor Navigate Search Project Run Window Help

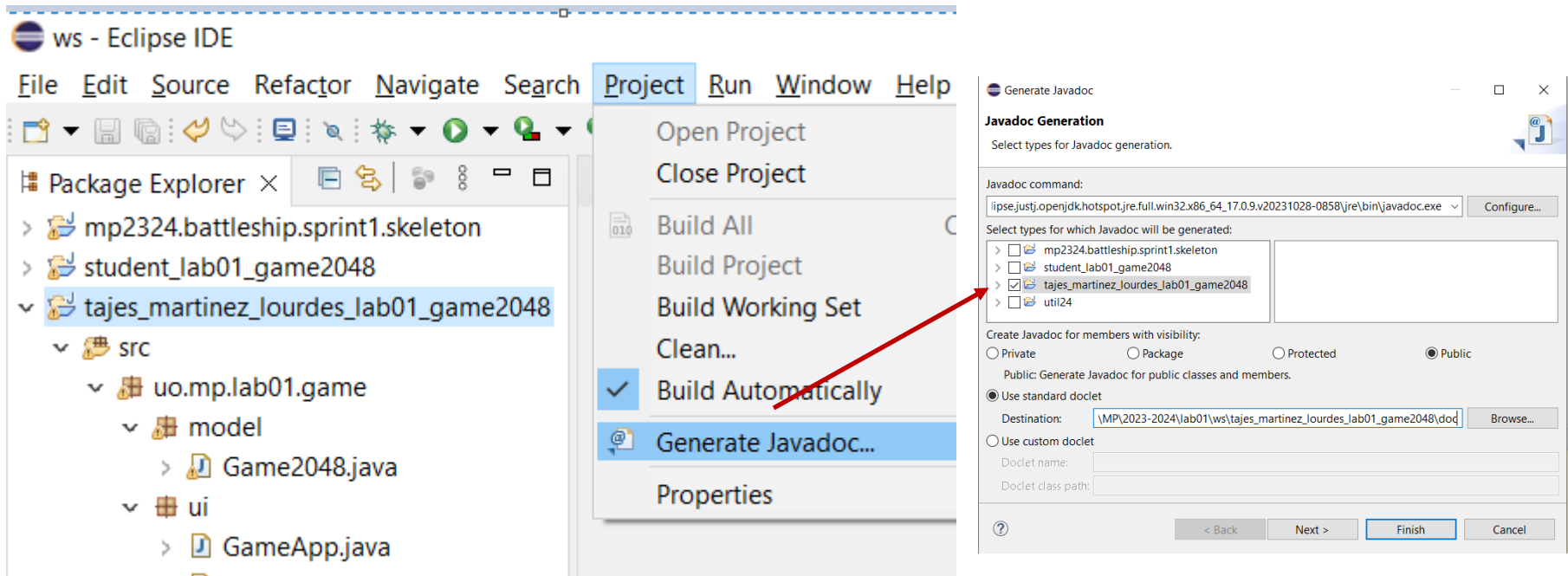
- Toggle Comment Ctrl+7
- Add Block Comment Ctrl+Shift+/
- Remove Block Comment Ctrl+Shift+\
- Generate Element Comment Alt+Shift+J
- Add Text Block Ctrl+Shift+'
- Shift Right
- Shift Left
- Correct Indentation Ctrl+I
- Format Ctrl+Shift+F

/**
 * @param user
 * @param computer
 */
public Game(HumanPlayer user, ComputerPlayer computer) {
 super();
 this.user = user;
 this.computer = computer;
}

Or write /** and hit enter

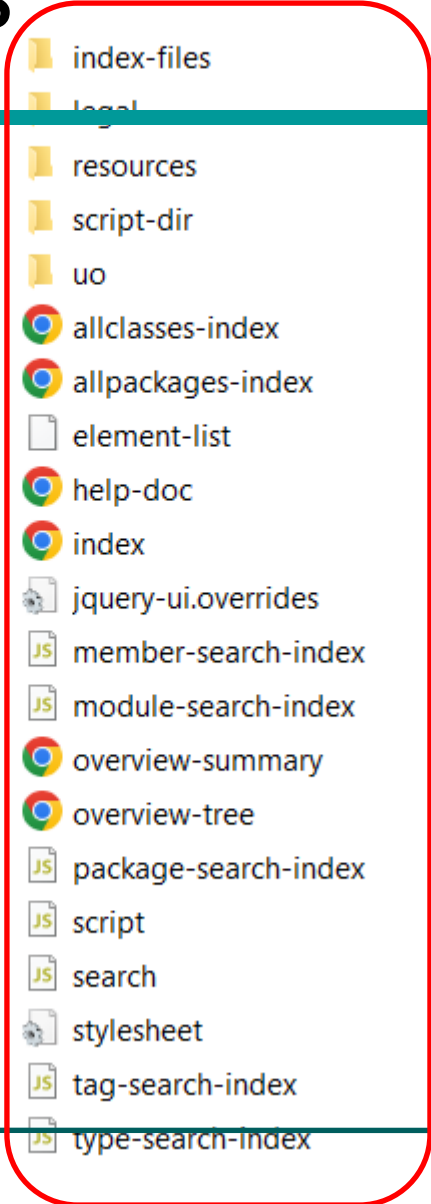
Generate Javadoc pages

- Select Project | Generate Javadoc...
- Then, use the project explorer to take a look to



Generate Javadoc pages

- Then, use the project explorer to take a look to



index-files
log4j
resources
script-dir
uo
allclasses-index
allpackages-index
element-list
help-doc
index
jquery-ui.overrides
member-search-index
module-search-index
overview-summary
overview-tree
package-search-index
script
search
stylesheet
tag-search-index
type-search-index

WRITE TESTS

Testing

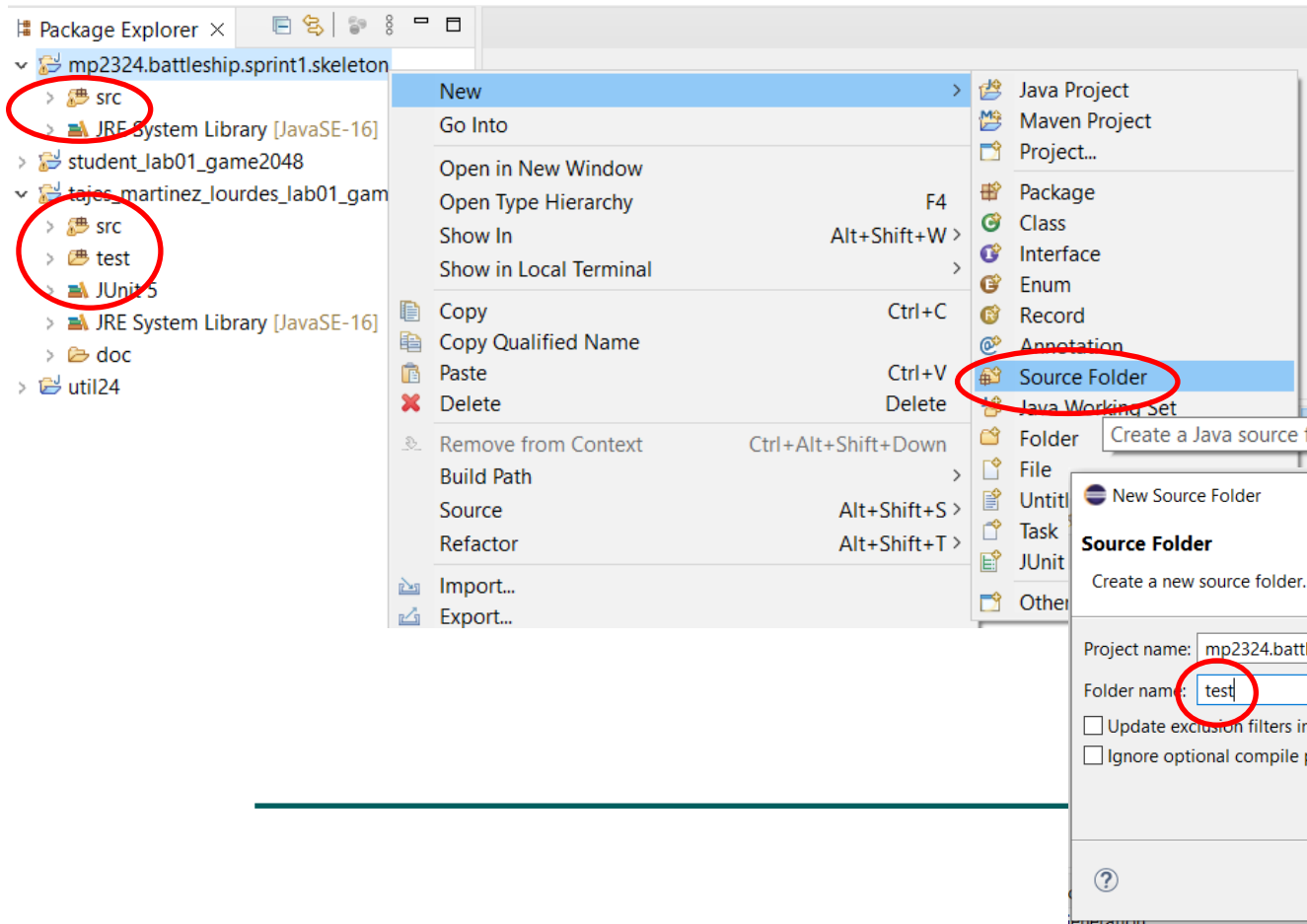
- Testing the software is **a way of providing information about its quality**
 - Testing **does not prove** software correctness
 - **Software correctness** can only be proven by means of **formal methods**
- There are different kinds of software tests
 - **Unit testing**
 - Integration testing
 - System testing
 - Regression testing
 - Acceptance testing (alpha, beta)
 - Performance testing
 - Usability testing
 - Security testing
 - Internationalization testing
 - Stress testing

Unit Testing

- We will focus on **unit testing**: Tests the functionality of a specific section (unit) of code
 - Commonly at the method (function) level
- **JUnit** a Java framework to obtain unit testing
- It performs unit **testing** of Java code **mechanically**
 - It can be used to perform **regression** testing when the source code is modified
- Written by Erich Gamma and Kent Beck
- Open Source <http://www.junit.org/>
- Fully integrated with the Eclipse IDE

Junit tests. Test folder

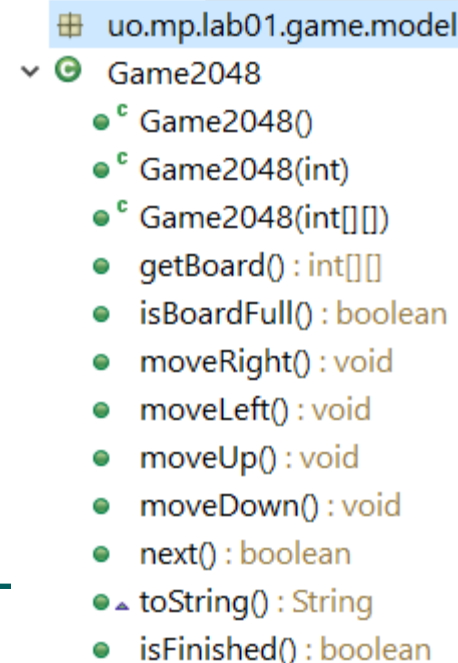
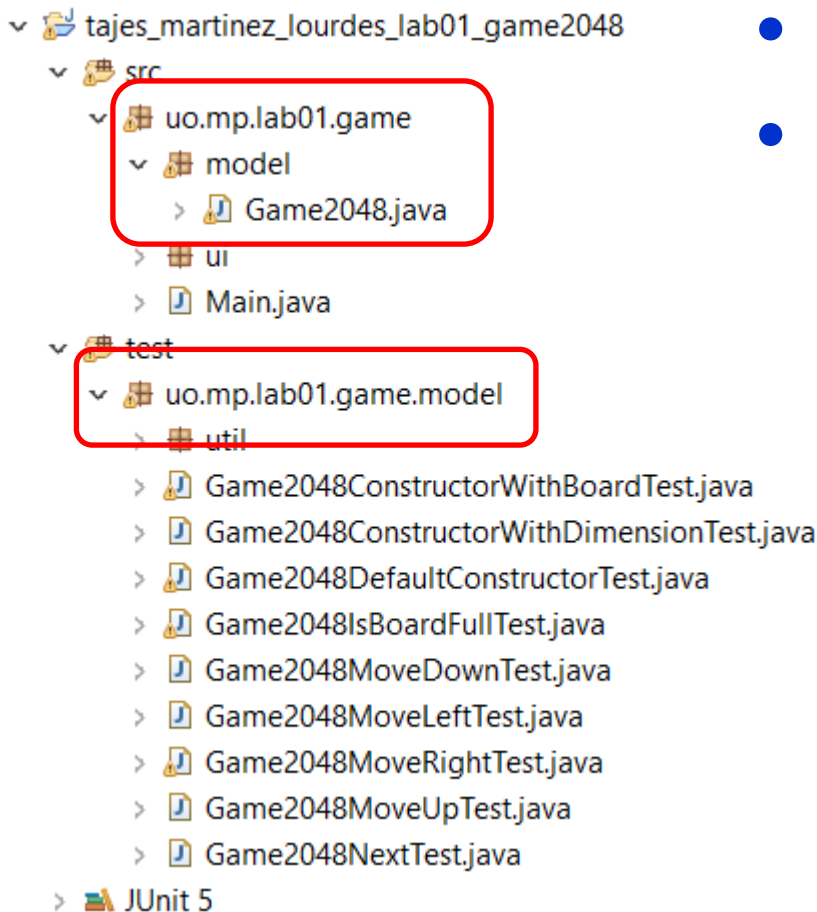
- Create a new **source folder** test in project battleship



JUnit tests. Test packages

- Same package structure
- One JUnit class for each method to test

className+methodName+Test



Test implementation

- Think about all tests to implement
 - Describe test scenarios
 - Write headers GIVEN WHEN THEN
 - Run them all and make them fail
- Implement them
- Run them all again and make them pass (green)

JUnit tests. Methods

```
11 /**
12  * Scenarios:
13  * - Fully filled board
14  * - Partially filled board
15  * - Empty board
16  */
17 @DisplayName("Test isBoardFull")
18 public class IsBoardFullTest {
19
20     @Test
21     @DisplayName("GIVEN: A full board")
22     /**
23      * GIVEN: a full board {{2,2,2},{2,2,2},{2,2,2}}
24      * WHEN: run is board full
25      * THEN: true
26      */
27     public void fullBoard() {
28
29         Game2048 game = new Game2048(ForTesting.FULL);
30         assertTrue(game.isBoardFull());
31     }
```

@BeforeEach

- It is pretty common that we require a **common initialization** for many tests
- If we annotate a method with **@BeforeEach**, it will be called before any **@Test** method

```
public class ClassTest {  
    private X a;  
    private X b;  
  
    @BeforeEach  
    public void setUp()  
        throws Exception {  
        a = new X("a");  
        b = new X("b");  
    }  
  
    @Test  
    public void testMethod() {  
        assertEquals("a", a.getName());  
        assertEquals("b", b.getName());  
    }  
}
```

@After

- Similarly, a method annotated with `@AfterEach` will be called by JUnit after executing any `@Test` method
- It is commonly used for releasing resources

Running Tests

Finished after 3,02 seconds

Runs: 5/5 Errors: 1 Failures: 1

uo.mp.lab01.game.model.Game2048WithoutPa

upperBound (0,002 s)

overBoundException (3,003 s)

otherBound (0,000 s)

lowerBound (0,000 s)

underBoundException (0,001 s)

```
    @Test(expected = IllegalArgumentException.class)
    public void overBoundException() {
        new Game2048(Game2048.MAX_SIZE+1000000000);
    }
```

Red cross indicates errors that arise if there is some unhandled exception in the test code

Blue cross indicates test failures that occur when you get different result from what you asserted for

```
@Test()
public void lowerBound() {
    int size = Game2048.MIN_SIZE;
    Game2048 g = new Game2048(size);
    int[][] board = g.getBoard();
    int[][] empty = {{Game2048.EMPTY_TILE, Game2048.EMPTY_TILE, Game2048.EMPTY_TILE},
                     {Game2048.EMPTY_TILE, Game2048.EMPTY_TILE, Game2048.EMPTY_TILE},
                     {Game2048.EMPTY_TILE, Game2048.EMPTY_TILE, Game2048.EMPTY_TILE}};
    assertTrue(board.length == size+9);
    assertTrue(Arrays.deepEquals(board, empty));
}
```

TO DO

General rules

- Submission must be done, as latest, **1 day before** your next laboratory session.
- **You might need them next session.**
- **You might use them in laboratory exams.**
- **All tasks must be uploaded to the corresponding task in CV.**
- If you fail to submit more than 2 tasks, you cannot pass the course **by continuous evaluation.**

To do

- Complete project game
- Rename the project
`surname1_surname2_name_lab01_game`
- Click on the element with the right button and select **Refactor/rename**
- There is a checklist for reviewing your project.
- Fill it and copy it inside the root of your project, before exporting